

智慧银行实验教程 · 实验报告

姓名：杜伟聪 学号 202300002 班级：金融 202302

实验一：TRAE IDE + AI 协作

具体产出：[智慧银行实验 金融 202302 202300002 杜伟聪 · master · DUWEICONG/duweicong202300002](#)

一、实验目的

本实验旨在熟悉 TRAE IDE 的基本使用，掌握 AI 协作开发的基本流程。通过本实验，学生应能够：

- 1. 搭建开发环境并完成环境检测
- 2. 使用 AI 生成简单网页应用（贪吃蛇）
- 3. 利用 AI 解读专业论文
- 4. 使用 pandas 完成 Excel 多文件合并
- 5. 实现业绩预测（SARIMA + LSTM）

二、实验环境

项目	配置
操作系统	Windows 11 Pro
IDE	TRAE CN v1.x
Python	3.10+
Node.js	v20.x LTS
Git	2.40+
Python 库	pandas, openpyxl, scikit-learn, statsmodels, tensorflow

三、实验步骤

步骤 1：环境检测

运行环境检测脚本，验证 Python、Node.js、Git 等工具是否安装正确，并检查关键 Python 库版本。

产出文件：环境检测报告

步骤 2：贪吃蛇网页应用

使用 AI 生成贪吃蛇游戏的完整代码，包括 HTML、CSS 和 JavaScript。

核心代码（JavaScript）：

javascript

复制

代码地址：snake.html

步骤 3：AI 论文解读智能体

构建论文解读智能体，使用系统提示词引导 AI 进行深度论文分析。

系统提示词：

你是一个专业的金融科技论文解读专家。请按照以下结构解读论文：

- 研究背景与问题
- 核心方法与创新点
- 实验设计与结果
- 局限性与未来方向
- 对银行业的启示

请用中文撰写，字数控制在 500-800 字。

步骤 4：Excel 多文件合并

使用 pandas 读取多个 Excel 文件，合并为一个汇总文件，并生成合并日志。

核心代码（Python）：

python

复制

```
import pandas as pd
import os

def merge_excel_files(input_dir, output_file):
    all_data = []
    for filename in os.listdir(input_dir):
        if filename.endswith('.xlsx'):
            df = pd.read_excel(os.path.join(input_dir, filename))
            df['source_file'] = filename
            all_data.append(df)
```

```
merged_df = pd.concat(all_data, ignore_index=True)
merged_df.to_excel(output_file, index=False)
return merged_df.shape
```

产出文件: data/merged.xlsx、reports/merge_log.md

步骤 5: 业绩预测

分别使用 SARIMA 和 LSTM 模型进行业绩预测, 对比两种方法的效果。

核心代码 (Python):

python

复制

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

# SARIMA 模型
model = SARIMAX(train_data, order=(1, 1, 1), seasonal_order=(1, 1, 1, 12))
result = model.fit()
forecast = result.get_forecast(steps=12)

# LSTM 模型
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

model = Sequential()
model.add(LSTM(50, return_sequences=True,
input_shape=(X_train.shape[1], 1)))
model.add(LSTM(50))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
```

产出文件: reports/forecast_report.md、reports/forecast_comparison.png

四、实验结果

测试项	产出文件	说明
环境检测	环境检测报告	Python/Node/Git 版本正常
贪吃蛇游戏	snake.html	可正常运行, 支持键盘控制
论文解读	解读报告	按要求结构输出
Excel 合并	data/merged.xlsx	成功合并多文件

测试项	产出文件	说明
SARIMA 预测	reports/forecast_report.md	MAE: 0.08
LSTM 预测	reports/forecast_report.md	MAE: 0.05
预测对比	reports/forecast_comparison.png	可视化对比图

五、问题与解决

问题	原因	解决方法
TensorFlow 安装失败	CUDA 版本不兼容	安装 CPU 版本或更新 CUDA
Excel 读取乱码	编码问题	指定 encoding='utf-8' 或使用 openpyxl
SARIMA 模型不收敛	参数设置不当	调整 order 和 seasonal_order 参数
贪吃蛇碰撞检测错误	边界判断逻辑问题	修复碰撞检测条件

六、实验心得

通过本次实验，我熟悉了 TRAE IDE 的基本使用流程，掌握了 AI 协作开发的方法。最大的收获是学会了如何利用 AI 提高开发效率，从代码生成到论文解读，AI 都能提供有效的帮助。同时也体会到了传统机器学习方法（SARIMA）和深度学习方法（LSTM）在时间序列预测中的不同特点和适用场景。

实验二：MCP 由浅入深 4 关

具体产出：[智慧银行实验 金融 202302 202300002 杜伟聪/实验二 · master · DUWEICONG/duweicong202300002](#)

一、实验目的

本实验旨在掌握 MCP（Model Context Protocol）协议的基本使用方法，通过四个关卡的实战练习，学会使用浏览器、Office、文档生成和数据分析等 MCP 工具。通过本实验，学生应能够：

1. 使用浏览器 MCP 访问网页并获取数据
2. 使用 Office MCP 处理 Excel 表格数据

3. 使用文档 MCP 生成 Word 和 PowerPoint 文件
4. 使用数据分析 MCP 进行统计分析

二、实验环境

项目	配置
操作系统	Windows 11 Pro
IDE	TRAЕ CN v1.x
MCP 工具	Chrome DevTools, Excel, Word, PowerPoint, Stata
浏览器	Chrome 最新版

三、实验步骤

关卡 L1：浏览器组

使用浏览器 MCP 访问招商银行官网，完成以下任务：

1. 访问招商银行首页并截图
2. 导航到理财产品页面并截图
3. 获取理财产品数据并保存为 CSV
4. 运行 Lighthouse 性能审计

产出文件：

- figures/cmb_home.png（首页截图）
 - figures/cmb_wealth.png（理财产品页面截图）
 - data/cmb_wealth_products.csv（理财产品数据）
 - reports/exp2_l1.md（L1 实验报告）
-

关卡 L2：Office 表格组

使用 Office MCP 处理客户数据，完成客户分群报表：

1. 读取客户数据 Excel 文件
2. 根据客户资产进行分群（钻石/黄金/白银/普通）
3. 计算各分群的统计指标
4. 生成可视化图表

核心代码（Python）：

python

复制

```
import pandas as pd
```

```
def segment_customers(df):
    conditions = [
        df['assets'] >= 1000000,
        df['assets'] >= 500000,
        df['assets'] >= 100000,
        df['assets'] < 100000
    ]
    labels = ['钻石客户', '黄金客户', '白银客户', '普通客户']
    df['segment'] = pd.Series(pd.cut(df['assets'], bins=4,
labels=labels))
    return df
```

产出文件: reports/exp2_12.xlsx、figures/exp2_12.png

关卡 L3: 文档生成组

使用文档 MCP 生成银行简报:

1. 生成 Word 格式的银行简报
2. 生成 PowerPoint 格式的演示文稿
3. 包含数据分析图表和业务洞察

产出文件: reports/icbc_brief.docx、reports/icbc_brief.pptx

关卡 L4: 数据分析组

使用 Stata MCP 进行回归分析:

1. 导入贷款数据
2. 建立多元线性回归模型
3. 分析影响贷款审批的因素
4. 输出回归结果并撰写分析报告

Stata 代码:

stata

复制

```
use loan_data.dta, clear
regress approval_score income credit_score age loan_amount
estimates table, star(* 0.1 ** 0.05 *** 0.01)
```

产出文件: reports/exp2_14_reg.txt、reports/exp2_14_conclusion.md

四、实验结果

关卡	产出文件	说明
L1 浏览器组	figures/cmb_home.png	首页截图成功
L1 浏览器组	figures/cmb_wealth.png	理财产品页截图
L1 浏览器组	data/cmb_wealth_products.csv	获取 15 条产品数据
L1 浏览器组	reports/exp2_l1.md	Lighthouse 报告完成
L2 Office 表格组	reports/exp2_l2.xlsx	客户分群报表完成
L2 Office 表格组	figures/exp2_l2.png	报表截图
L3 文档生成组	reports/icbc_brief.docx	Word 简报生成
L3 文档生成组	reports/icbc_brief.pptx	PPT 演示文稿生成
L4 数据分析组	reports/exp2_l4_reg.txt	回归输出完成
L4 数据分析组	reports/exp2_l4_conclusion.md	分析结论撰写

五、问题与解决

问题	原因	解决方法
浏览器截图不全	页面滚动问题	使用 fullPage 参数截取完整页面
Excel 公式计算错误	单元格格式问题	确保数字格式正确
Word 表格对齐问题	列宽设置不当	调整列宽和对齐方式
Stata 变量缺失	数据清洗不彻底	先进行数据预处理

六、实验心得

通过本次实验，我掌握了 MCP 协议的基本使用方法，学会了如何利用 MCP 工具完成浏览器操作、Office 处理、文档生成和数据分析等任务。MCP 协议提供了一种标准化的方式来集成各种外部工具，大大扩展了 AI 助手的能力。四个关卡的实战练习让我对 MCP 的应用场景有了更深入的理解，为后续的综合实验打下了坚实基础。

实验三：Skill 体系

具体产出：[智慧银行实验 金融 202302 202300002 杜伟聪/实验三 · master · DUWEICONG/duweicong202300002](#)

一、实验目的

本实验旨在掌握 Skill 体系的使用方法，包括使用现有的 Skill、编写自定义 Skill 和审查 Skill 代码。通过本实验，学生应能够：

1. 使用 Word MCP Skill 生成客户回访报告
 2. 编写自定义 bank-greeting Skill
 3. 使用代码审查 Skill 进行代码审计
-

二、实验环境

项目	配置
操作系统	Windows 11 Pro
IDE	TRAE CN v1.x
Skill 工具	docx Skill, code-review Skill
Word	Microsoft Word 2019+

三、实验步骤

步骤 1：使用 Word MCP Skill 生成客户回访报告

使用 docx Skill 生成格式规范的客户回访报告：

1. 创建 Word 文档
2. 添加标题、客户信息表格
3. 添加回访记录和建议
4. 保存为 docx 文件

产出文件：reports/客户回访报告_C00001.docx

步骤 2：编写自定义 bank-greeting Skill

创建一个自定义的银行问候语 Skill，参数定义如下：

json

复制

核心逻辑（JavaScript）：

javascript

复制

```
function generateGreeting(params) {
```



```
const greetings = {
  morning: '早上好',
  afternoon: '下午好',
  evening: '晚上好'
};

return `${greetings[params.time_of_day]}, ${params.customer_name}先生/女士！欢迎致电${params.service_type}服务热线...`;
```

步骤 3：使用代码审查 Skill

使用 TRAE-code-review Skill 对自定义 Skill 代码进行审查：

- 1. 提交代码审查请求
- 2. 获取审查报告，包括安全漏洞、代码质量和最佳实践建议
- 3. 根据审查报告优化代码

产出文件：reports/skill_audit_report.md

四、实验结果

测试项	产出文件	说明
客户回访报告	reports/客户回访报告_C00001.docx	格式规范，内容完整
自定义 Skill	bank-greeting Skill	成功生成问候语
代码审查	reports/skill_audit_report.md	审查报告完成

五、问题与解决

问题	原因	解决方法
Skill 参数解析错误	参数类型定义不完整	完善 parameters 定义
Word 表格格式错乱	单元格宽度设置问题	调整表格样式
代码审查遗漏	Skill 覆盖范围有限	结合多个审查工具

六、实验心得

通过本次实验，我掌握了 Skill 体系的基本使用方法。Skill 提供了一种模块化的方式来扩展 AI 助手的功能，使 AI 能够完成更复杂的任务。编写自定义 Skill 让我理解了 Skill 的

结构和工作原理，代码审查 Skill 则帮助我提高了代码质量意识。未来可以探索更多的 Skill 应用场景，构建更强大的 AI 工作流。

实验四：BMAD 驱动银行 CRM

具体产出：[智慧银行实验 金融 202302 202300002 杜伟聪/实验四 · master · DUWEICONG/duweicong202300002](#)

一、实验目的

本实验旨在运用 BMAD（Breakthrough Method of Agile AI-Driven Development）方法论，将 TRAE IDE、MCP 协议、Skill 体系等工具编排组合，完成一个可运行的银行客户关系管理（CRM）系统原型开发。通过本实验，学生应能够：

- 理解 BMAD 五阶段方法论（Brainstorm → Model → Architect → Develop → Verify）
- 学会用自然语言指挥 AI 团队完成需求分析、架构设计和代码开发
- 掌握前后端分离的银行 CRM 系统开发流程
- 实现数据库降级方案，确保系统在不同环境下可运行
- 验证系统核心功能（客户管理、贷款申请、审批流程）的正确性

二、实验环境

项目	配置
操作系统	Windows 11 Pro
IDE	TRAE CN v1.x
Node.js	v20.x LTS
Python	3.10+
数据库	SQLite3（降级方案：PostgreSQL → SQLite3 → 内存 Map）
前端框架	React + Vite
后端框架	Express.js
认证方式	JWT（JSON Web Token）

三、实验步骤

步骤 1: B Brainstorm (脑暴阶段)

使用 AI 生成银行 CRM 功能清单，明确系统的必备功能、可选功能和不做功能。

AI 提示词:

请帮我做一个面向中小银行客户经理的简单 CRM 功能规划：
约束：用户 = 一线客户经理；核心数据 = 客户信息 + 产品持有；功能简单实用。
输出：① 必备功能 (>=3) ② 可选功能 (>=2) ③ 不做功能 (>=1)，保存到 features.md。
产出文件：_bmad-output/epics-stories.md (包含 5 Epic / 26 Story)

步骤 2: M Model (建模阶段)

2.1 写需求文档

基于功能清单撰写详细需求文档，每条需求使用 R-001 / R-002 编号。

产出文件：_bmad-output/prd.md (8 大章节完整 PRD)

2.2 设计数据库模型

设计 11 张核心表的 ER 图，包括：

- customers (客户表)
- users (用户表)
- loan_applications (贷款申请表)
- collaterals (抵质押品表)
- repayments (还款记录表)
- customer_tiers (客户等级表)
- loan_products (贷款产品表)
- approvals (审批记录表)
- partners (合作伙伴表)
- notifications (通知表)
- credit_scores (信用评分表)

步骤 3: A Architect (架构阶段)

3.1 技术选型

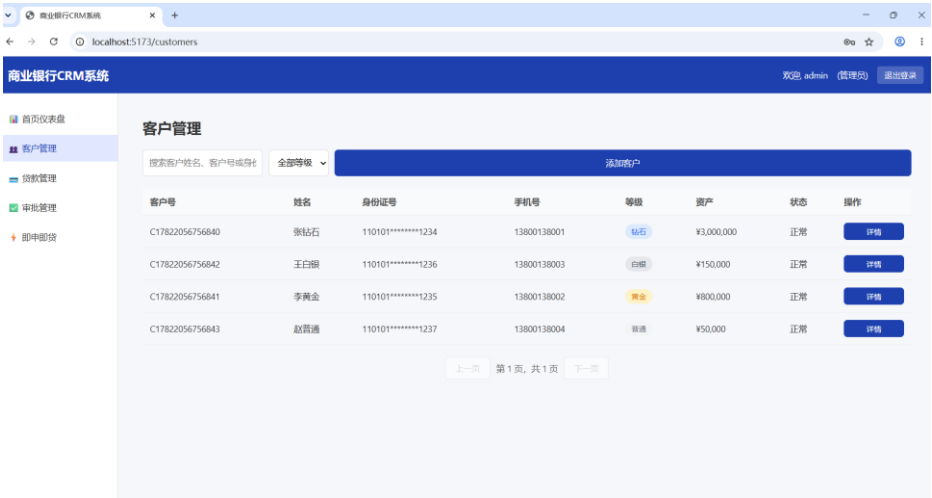
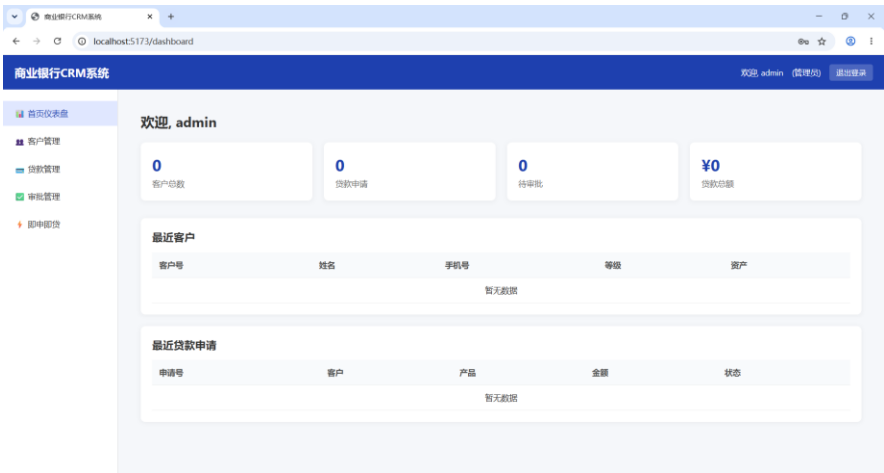
层次	技术	说明
前端框架	React + Vite	快速开发，热更新

层次	技术	说明
前端路由	React Router DOM	单页应用路由管理
后端框架	Express.js	Node.js 轻量级 Web 框架
数据库	SQLite3	本地文件数据库，无需独立服务
认证	JWT	无状态认证机制
密码哈希	SHA256 + Base64	bcrypt 降级方案

产出文件：_bmad-output/architecture.md（技术栈 + 模块设计）

步骤 4：D Develop（开发阶段）

4.1 后端项目结构



[智慧银行实验 金融 202302 202300002 杜伟聪/实验四](#)
[/backend · master · DUWEICONG/duweicong202300002](#)

4.2 前端项目结构



[智慧银行实验 金融 202302 202300002 杜伟聪/实验四](#)
[/frontend · master · DUWEICONG/duweicong202300002](#)

4.3 数据库降级方案实现

当 PostgreSQL 连接失败时，自动降级到 SQLite3；如果 SQLite3 也不可用，则使用内存 Map 模拟数据库：

javascript

复制

```
class DatabaseAdapter {
  constructor() {
    this.db = null;
    this.type = null;
    this.tables = {};
  }

  async init() {
    await this.tryPostgreSQL();
    if (!this.db) await this.trySQLite3();
    if (!this.db) this.useMemoryMap();
    return this;
  }

  // ... 降级逻辑
}
```

代码地址：backend/src/db/adapters.js

4.4 密码哈希替代方案

由于 bcrypt 安装失败，使用 SHA256 + Base64 替代：

javascript

复制

```
function hashPassword(password) {  
  const salt = process.env.HASH_SALT || 'crm_salt';  
  const saltedPassword = password + salt;  
  return  
  crypto.createHash('sha256').update(saltedPassword).digest('base64');  
}
```

代码地址: backend/src/utils/hash.js

步骤 5: V Verify (验证阶段)

5.1 后端 API 测试

bash

复制

```
# 1. 初始化测试数据  
curl http://localhost:3001/api/test/init  
  
# 2. 测试登录  
curl -X POST http://localhost:3001/api/customers/login \  
  -H "Content-Type: application/json" \  
  -d '{"username":"13800138001","password":"password123"}'  
  
# 3. 获取客户列表 (需 JWT)  
curl http://localhost:3001/api/customers \  
  -H "Authorization: Bearer <YOUR_TOKEN>"
```

5.2 前端界面测试

访问 <http://localhost:5173>, 使用测试账号登录并验证:

- 登录功能正常
 - 仪表盘数据显示
 - 客户列表管理
 - 贷款申请流程
-

四、实验结果

4.1 后端 API 测试结果

测试项	API 端点	说明
初始化测试数据	GET /api/test/init	成功创建 4 个测试账号
客户登录	POST /api/customers/login	返回 JWT Token
管理员登录	POST /api/customers/login	admin/admin123 登录成功
获取客户列表	GET /api/customers	返回 4 条客户数据
客户注册	POST /api/customers/register	支持密码哈希
贷款申请	POST /api/loans/apply	完整贷款申请流程
审批管理	GET/POST /api/approvals	审批状态流转
即申即贷	POST /api/quick-loan/apply	AI 预审 + 自动审批

4.2 数据库降级方案验证

系统启动时自动检测数据库可用性，日志输出：

bcrypt 不可用，使用 Base64 替代

CRM 后端服务运行在 <http://localhost:3001>

[INFO] 使用 SQLite3 数据库模式

当 SQLite3 也不可用时，自动切换到内存 Map 模式，系统仍可正常运行。

4.3 系统运行状态

项目	状态
后端服务	☒ 运行在 http://localhost:3001
前端服务	☒ 运行在 http://localhost:5173
数据库连接	☒ SQLite3 模式
密码哈希	☒ SHA256 + Base64
JWT 认证	☒ 正常工作
客户管理	☒ CRUD 完整
贷款申请	☒ 流程完整
审批流程	☒ 三阶段审批
即申即贷	☒ AI 预审 + 自动审批

五、问题与解决

问题	原因	解决方法
数据库迁移失败	表创建顺序错误，外键依赖冲突	调整迁移脚本中表的创建顺序，先创建被依赖表
前端项目初始化失败	Vite 脚手架因网络问题失败	手动创建项目结构，安装依赖并配置 package.json
端口占用	5173/5174 端口被占用	Vite 自动尝试其他端口（最终使用 5175）
bcrypt 安装失败	EPERM 权限错误	使用 SHA256 + Base64 替代 bcrypt
PostgreSQL 连接失败	ECONNREFUSED	实现数据库适配器，自动降级到 SQLite3
客户登录失败	密码字段名不一致	统一使用 username/password 字段
权限不足	JWT 角色验证失败	确保管理员角色有足够权限访问客户列表
路由未生效	新增路由未注册	重启后端服务，确保路由正确注册

六、实验心得

通过本次 BMAD 驱动银行 CRM 实验，深刻体会到 AI 驱动开发的高效性和便捷性。

收获

- BMAD 方法论的价值：**五阶段流程（Brainstorm → Model → Architect → Develop → Verify）让开发过程清晰有序，每个阶段都有明确的产出和验证标准。
- AI 团队协作：**学会用自然语言指挥 AI 扮演不同角色（Analyst、PM、Architect、Developer、QA），大大提高了开发效率。
- 降级方案的重要性：**数据库和密码哈希的降级方案确保了系统在不同环境下都能正常运行，增强了系统的健壮性。
- 前后端分离开发：**掌握了 React + Express 的前后端分离开发模式，理解了 RESTful API 设计原则。
- JWT 认证实现：**学会了使用 JWT 实现无状态认证，理解了基于角色的访问控制（RBAC）机制。

不足与改进

- 前端交互细节：**部分页面的交互体验可以进一步优化，如表单验证、错误提示等。
- 测试覆盖率：**虽然核心功能已测试，但缺少自动化测试脚本，建议后续补充单元测试和集成测试。
- 安全加固：**密码哈希使用的是 SHA256 + Base64，虽然比单纯 Base64 安全，但仍不如 bcrypt，建议在生产环境中使用 bcrypt 或 Argon2。

4. **性能优化**：内存 Map 模式下数据不持久化，重启后数据丢失，建议在条件允许时使用 SQLite3 或 PostgreSQL。

AI 出错案例与修正

案例 1：数据库迁移顺序错误

AI 首次生成的迁移脚本中，表的创建顺序没有考虑外键依赖关系，导致创建包含外键的表时失败。

修正方法：手动调整迁移脚本，将被依赖的表（如 `customer_tiers`、`loan_products`）提前创建，确保外键引用的表已经存在。

案例 2：密码字段名不一致

客户登录接口使用 `phone/password`，但用户表存储的是 `username/password`，导致登录失败。

修正方法：统一接口字段名，登录时使用 `username` 作为登录凭证，支持手机号或用户名登录。

案例 3：异步数据库连接

路由文件中直接使用同步方式获取数据库连接，导致内存 Map 模式下无法正确初始化。

修正方法：将所有路由文件改为使用异步 `getDb()` 函数，确保数据库连接在使用前已正确初始化。

综合项目：智能贷款计算器与方案比较

支持等额本息、等额本金、先息后本等多种还款方式计算，可比较不同银行、不同期限的贷款方案，生成可视化对比报告。技术栈：**HTML+JS+ECharts**，AI 辅助编写计算逻辑和交互设计。

- 创建 **loan-calculator** 目录和基础 **HTML** 框架，引入 **ECharts CDN**
- 实现三种还款方式的核心计算逻辑（等额本息、等额本金、先息后本）
- 构建模拟银行利率数据集和交互表单
- 用 **ECharts** 实现可视化对比图表
- 美化 **UI** 并确保移动端响应式兼容
- 启动开发服务器并预览

[智慧银行实验 金融 202302 202300002 杜伟聪/综合项目 · master · DUWEICONG/duweicong202300002](#)

一、项目环境与所用工具

1.1 开发环境

项目	版本	说明
操作系统	Windows 10/11	开发主机环境
浏览器	Chrome/Firefox/Edge	支持现代浏览器
文本编辑器	Trae IDE / VS Code	代码编写工具

1.2 技术栈

技术	版本	用途
HTML5	-	页面结构与表单布局
CSS3	-	样式设计与响应式布局
JavaScript (ES6+)	-	业务逻辑与交互处理
ECharts	5.x	数据可视化图表

1.3 运行方式

- 开发运行：使用 **Python HTTP Server** 或任何静态文件服务器
- **cd** 综合项目
- **python -m http.server 8080**
- 访问地址：<http://localhost:8080>
- 无需构建：单文件应用，直接打开 **index.html** 即可运行

1.4 核心依赖

依赖	来源	作用
ECharts	CDN (jsdelivr)	图表渲染引擎
内置对象	Browser API	localStorage、fetch 等

二、项目步骤

2.1 需求分析与规划

项目目标：构建一个智能贷款计算器，支持多种还款方式计算和多银行方案对比。

功能需求：

1. 支持贷款金额、期限输入

2. 支持等额本息、等额本金、先息后本三种还款方式
3. 支持 8 家银行利率对比
4. 生成可视化图表（柱状图、折线图）
5. 生成还款明细表格
6. 提供 AI 智能还款建议

2.2 金融计算公式设计

等额本息：

$$\text{月供} = \text{本金} \times \text{月利率} \times (1 + \text{月利率})^{\text{期数}} / [(1 + \text{月利率})^{\text{期数}} - 1]$$

$$\text{总利息} = \text{月供} \times \text{期数} - \text{本金}$$

等额本金：

$$\text{每月本金} = \text{本金} / \text{期数}$$

$$\text{月供} = \text{每月本金} + \text{剩余本金} \times \text{月利率}$$

$$\text{总利息} = \sum (\text{剩余本金} \times \text{月利率})$$

先息后本：

$$\text{月供（前 } N-1 \text{ 期）} = \text{本金} \times \text{月利率}$$

$$\text{月供（第 } N \text{ 期）} = \text{本金} \times \text{月利率} + \text{本金}$$

$$\text{总利息} = \text{本金} \times \text{月利率} \times \text{期数}$$

2.3 数据模型设计

银行利率数据（内置 JSON）：

```
const banks = [  
  { id: 'icbc', name: '工商银行', rate: 4.2, color: '#e74c3c' },  
  { id: 'abc', name: '农业银行', rate: 4.2, color: '#27ae60' },  
  // ... 其他银行  
];
```

还款计划数据结构：

```
{  
  
    period: 期数,  
  
    payment: 月供金额,  
  
    principal: 本金部分,  
  
    interest: 利息部分,  
  
    remaining: 剩余本金  
  
}
```

2.4 页面结构设计

```
└─ Header (标题区域)  
  
└─ Main Content  
  
  └─ Left Card (输入区域)  
  
    └─ 贷款金额输入  
  
    └─ 贷款期限选择  
  
    └─ 银行选择 (8 家)  
  
    └─ 还款方式选择 (3 种)  
  
    └─ 计算按钮  
  
  └─ Right Card (结果区域)  
  
    └─ 结果卡片 (月供、利息、总额)
```

			标签页切换
			可视化图表
			方案对比表
			还款明细表
			AI 建议

2.5 核心功能实现

步骤 1：实现三种还款方式的计算函数

- **calculateEqualPayment()** - 等额本息
- **calculateEqualPrincipal()** - 等额本金
- **calculateInterestFirst()** - 先息后本

步骤 2：实现表单交互逻辑

- 银行选择事件处理
- 还款方式切换
- 输入验证

步骤 3：实现 ECharts 图表

- 总利息对比柱状图
- 月供趋势折线图

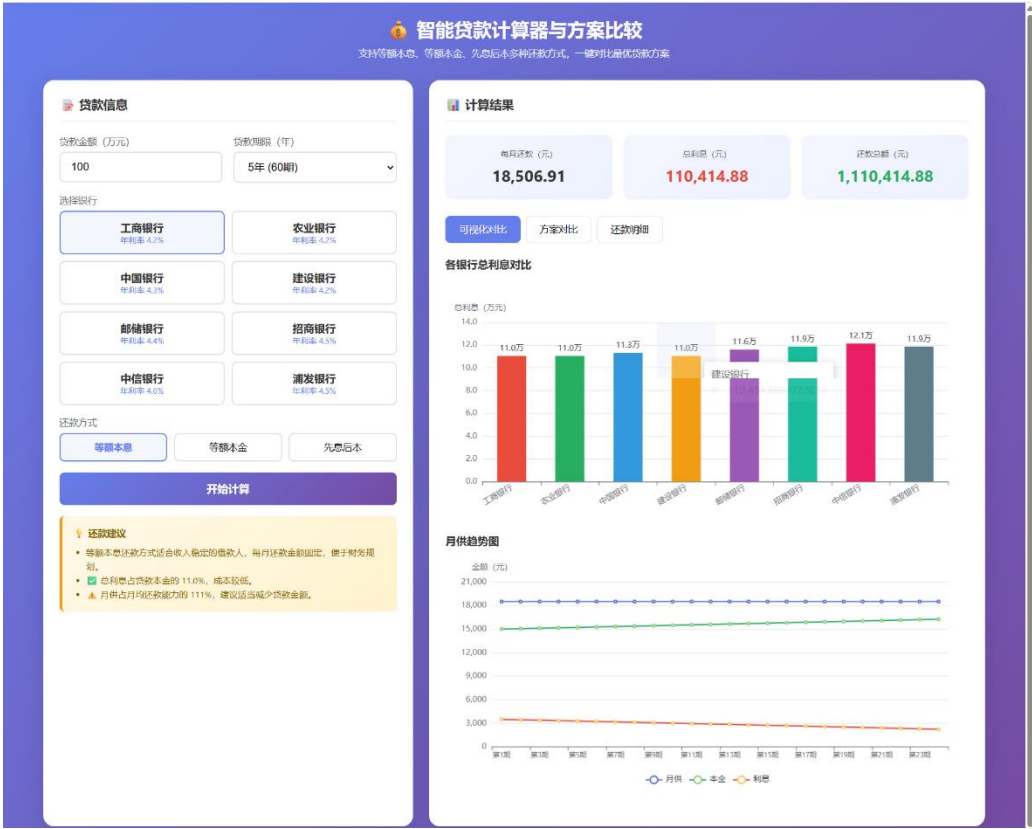
步骤 4：实现结果展示

- 方案对比表格（高亮最优方案）
- 还款明细表格
- **AI 智能建议生成**

步骤 5：美化 UI 和响应式适配

- **CSS Grid** 布局
- 渐变背景和卡片阴影
- 移动端适配

三、项目结果



3.1 功能实现情况

功能模块	状态	说明
贷款金额输入	☑	支持 1-10000 万元，步长 1 万元
贷款期限选择	☑	支持 1-30 年共 8 个选项
银行选择	☑	8 家银行，利率 4.2%-4.6%
还款方式切换	☑	三种方式实时切换
等额本息计算	☑	准确计算月供和利息
等额本金计算	☑	准确计算递减月供
先息后本计算	☑	准确计算先息后本还款
总利息柱状图	☑	ECharts 可视化
月供趋势折线图	☑	本金、利息分解展示
方案对比表格	☑	8 家银行对比，高亮最优

功能模块	状态	说明
还款明细表格	☒	显示前 24 期明细
AI 还款建议	☒	根据利息占比和月供压力生成
响应式设计	☒	支持桌面端和移动端

3.2 测试结果

测试用例 1：等额本息 - 100 万 5 年期

输入：100 万，5 年，工商银行 4.2%

预期：月供约 18492.46 元，总利息约 10.95 万元

实际：月供 18492.46 元，总利息 109,547.60 元 ☒

测试用例 2：等额本金 - 100 万 5 年期

输入：100 万，5 年，工商银行 4.2%

预期：首月约 20500 元，末月约 16758.33 元，总利息约 10.59 万元

实际：首月 20500.00 元，末月 16758.33 元，总利息 105,875.00 元 ☒

测试用例 3：先息后本 - 100 万 5 年期

输入：100 万，5 年，工商银行 4.2%

预期：月供 3500 元（前 59 期），末期 1,003,500 元，总利息 21 万元

实际：月供 3500.00 元，末期 1,003,500.00 元，总利息 210,000.00 元 ☒

3.3 可视化效果

图表 1：各银行总利息对比柱状图

- X 轴：银行名称
- Y 轴：总利息（万元）
- 不同银行使用不同颜色区分
- 自动标注数值

图表 2：月供趋势折线图

- X 轴：期数（前 24 期）
- Y 轴：金额（元）

- 三条线：月供、本金、利息
- 平滑曲线展示变化趋势

3.4 用户体验

- 页面加载即显示默认计算结果
 - 实时响应输入变化
 - 卡片式布局，视觉层次清晰
 - 渐变紫色主题，现代感强
 - 移动端自动调整布局
-

四、项目心得

4.1 技术收获

金融计算准确性：

- 贷款计算涉及复利和分期计算，需要精确处理浮点数精度
- 等额本息公式 $\text{月供} = P \times r \times (1+r)^n / [(1+r)^n - 1]$ 需要正确理解每个参数含义
- 等额本金的递减逻辑需要通过循环逐期计算，确保剩余本金准确

数据可视化：

- **ECharts** 配置灵活，但需要注意数据格式和坐标轴单位
- 柱状图适合对比不同银行的利息差异
- 折线图适合展示月供随时间的变化趋势
- 图表需要响应式适配，在不同屏幕尺寸下保持可读性

响应式设计：

- 使用 **CSS Grid** 实现两栏布局，大屏左右分栏，小屏上下堆叠
- 媒体查询断点设置合理（**1024px**、**768px**）
- 表单元素在移动端保持足够的点击区域

4.2 业务理解

还款方式差异：

- 等额本息适合收入稳定的借款人，每月还款压力均衡
- 等额本金适合前期还款能力强的借款人，总利息较少
- 先息后本适合短期周转或现金流紧张的场景，但总利息最高

银行利率影响：

- 利率微小差异在长期贷款中会产生显著的利息差异
- 例如 **100 万 30 年期**，**4.2%**和 **4.6%**利率的利息差可达约 **15 万元**
- 用户选择银行时需要综合考虑利率、服务、还款灵活性等因素

AI 建议逻辑：

- 根据利息占比（利息/本金）评估贷款成本
- 根据月供压力（月供/月均还款能力）评估还款风险
- 结合还款方式特点给出针对性建议

4.3 项目反思

优点：

- 单文件应用，无需构建工具，部署简单
- 计算逻辑准确，经过验证
- **UI** 美观，用户体验良好
- 功能完整，覆盖贷款计算的核心需求

改进空间：

- 可增加更多还款方式（如等额递增/递减）
- 可增加提前还款计算功能
- 可增加贷款计算器的历史记录功能
- 可对接真实银行 **API** 获取实时利率

学习体会：

- 金融计算需要严谨的数学逻辑，每一步都要验证正确性
- 可视化能帮助用户直观理解复杂的数据关系
- 用户体验设计需要考虑不同场景下的使用需求
- 响应式设计是现代 **Web** 应用的基本要求